

Module 3

Bitcoin techniques

How to store and use Bitcoins

The important preconditions to use Bitcoins are:

- be able to spend your own money when you want to
- nobody else can spend your money
- convenience: the process must be relatively simple and don't cost too much

Possible approaches:

- **+ convenience**: the money could be accessible just pushing a button
- **- availability**: if the device crashes or you lose it, the money is lost with it
- **- security**: if someone manages to break into the device the security is broken. It's just as safe as carrying money in the pocket.

1. Storing Bitcoins

- **Hot Wallets (Online Storage)**

Examples: Mobile wallets, desktop wallets, web wallets.

Popular Hot Wallets:

- **Mobile Wallets:** Trust Wallet, Exodus, Mycelium.
- **Desktop Wallets:** Electrum, Bitcoin Core.
- **Web Wallets:** Blockchain.com, Coinbase Wallet

Cold Wallets (Offline Storage)

- **Hardware Wallets:**



- Paper wallet



- **Air-Gapped Devices**
- **Brain wallet**
encrypt info under passphrase that user remembers

C. Multisignature Wallets (Multisig)

-- Example: Casa, Electrum with multisig setup.

Online wallets

Advantages:

- it is not necessary to install software to use a web based wallet, or a simple app on the phone

- it works across multiple devices. Even if a device is lost, the wallet will still be available

Disadvantages:

- if the website or app is malicious or gets compromised somehow, the Bitcoins can be lost. It is necessary to trust the website as more secure than oneself.

Summary

Hot Storage:

- Connected to the internet
- Higher risk of hacking and theft
- - Used for frequent transactions

Cold Storage:

- - Not connected to the internet
- - Lower risk of hacking and theft
- - Used for long-term storage and security

This strategy balances the ease of access provided by hot wallets with the robust security of cold storage.

2.Using Bitcoins

a. Making Transactions

Sending Bitcoin:

- Use a wallet app to send Bitcoin to a recipient's public address.
- Verify the address and transaction amount before confirming.

Receiving Bitcoin:

- Share your public wallet address or QR code to receive funds.
- Wait for blockchain confirmations (typically 6 confirmations for full security).

b. Spending Bitcoin

--**Online Retailers:** Many merchants accept Bitcoin for goods and services (e.g., Microsoft, Overstock, Shopify stores).

--**Gift Cards:** Platforms like Bitrefill allow Bitcoin to be used for purchasing gift cards.

--**Travel: Companies** like Travala and Expedia support Bitcoin payments for flights

c. Trading Bitcoin

- Use cryptocurrency exchanges like Binance, Coinbase, or Kraken to trade Bitcoin for fiat currency or other cryptocurrencies.
- Leverage decentralized exchanges (DEXs) for peer-to-peer transactions.

3. Security Best Practices

To protect your Bitcoin, adhere to the following practices:

- **Backup Your Wallet:**

- Create backups of your wallet seed phrases or private keys.
- Store them in a secure, offline location like a fireproof safe.

- **Use Strong Passwords:**

- Ensure wallets and accounts are secured with complex, unique passwords.

- **Enable Two-Factor Authentication (2FA):**

- Add an extra layer of security for accessing your wallets and accounts.

- **Be Wary of Phishing Attempts:**

- Avoid clicking on suspicious links or sharing your private keys.

- **Regular Software Updates:**

- Keep your wallet apps and hardware devices updated to protect against vulnerabilities.

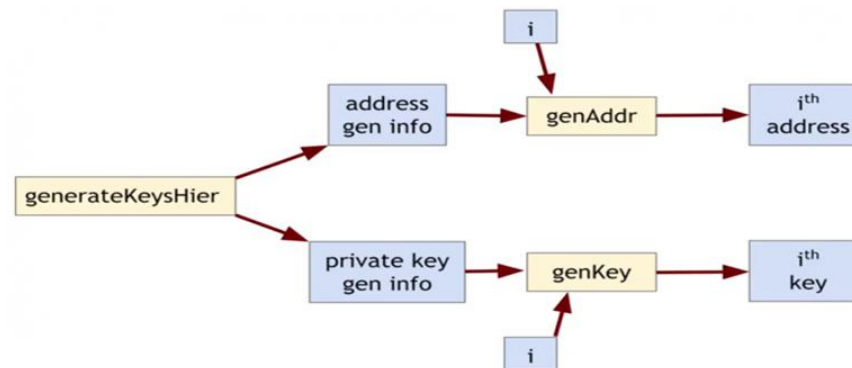
Hierarchical key generation:

- This method is particularly relevant to the structure of Hierarchical Deterministic (HD) wallets, as outlined in Bitcoin Improvement Proposal 32 (BIP-32).

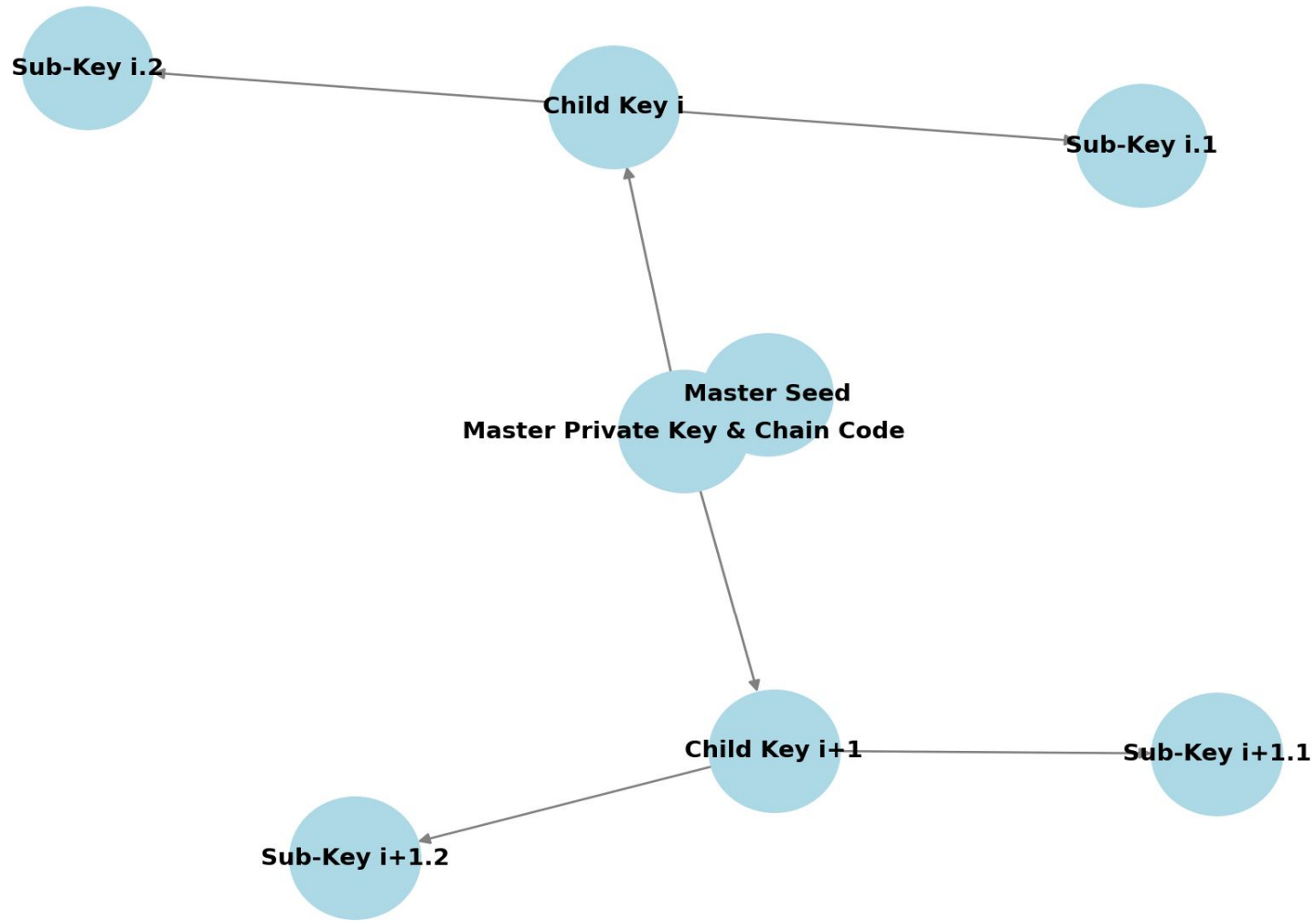
This approach offers several advantages:

- Simplified Key Management
- Enhanced Security
- Organizational Structure

Businesses can assign different levels of access and control by distributing derived keys to employees or departments, aligning with organizational hierarchies



Hierarchical Key Generation (Mapping with Indices i and $i+1$)



Formula for Child Key Derivation:

$$I = \text{HMAC-SHA512}(\text{Parent Chain Code}, \text{Parent Key})$$

Where:

- I is split into two halves:
 - Left half = Child Private/Public Key
 - Right half = Child Chain Code
- This ensures that each child key has its own chain code

Hierarchical key generation in Bitcoin and other cryptocurrencies provides a robust framework for key management, enhancing both security and usability for individuals and organizations.

Problem:

Want to use a new address (and key) for each coin sent to cold

But how can hot wallet learn new addresses if cold wallet is offline?

Better solution:

A **Hierarchical Deterministic (HD) Wallet** solves this issue by generating multiple addresses from a single master key.

• How It Works:

1. Master Key (Seed):

- The cold wallet generates a **master private key** (seed).
- From this seed, a **hierarchical tree** of public/private key pairs can be derived.

2. Public Key Derivation for the Hot Wallet:

- The cold wallet **derives a master public key** from the master private key.
- The **master public key** (not the private key) is shared with the hot wallet.

3. Hot Wallet Generates New Addresses:

- The hot wallet can now generate new public addresses **without needing the private key**.
- Every time a transaction is made, the hot wallet can derive a **new address** from the public key tree.
- Since the **cold wallet holds the master private key**, it can always derive the corresponding private key when needed.

4. Offline Security Maintained:

- The cold wallet stays offline, keeping private keys secure.
- The hot wallet only has access to **public keys** and cannot spend coins

Conclusion:

A hierarchical wallet enables secure, scalable, and automated management of addresses for cryptocurrency transactions while keeping the cold wallet offline and hot wallet online for transaction processing.

- **Split and share keys**

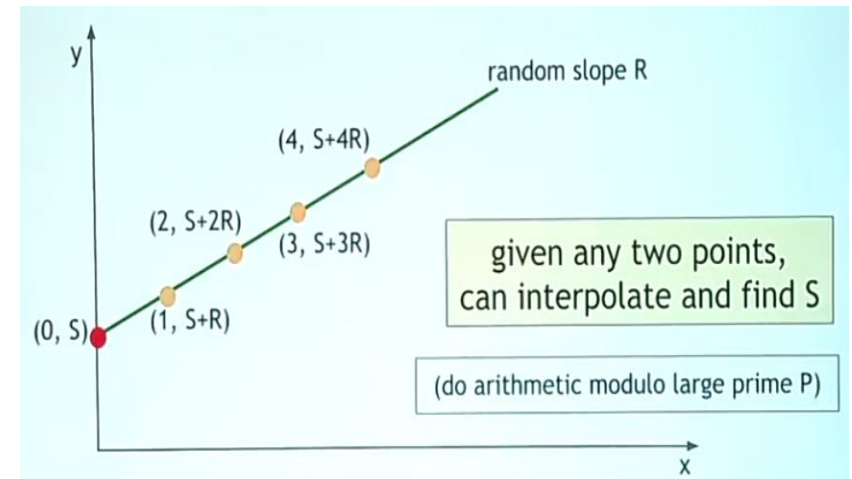
How to increase N , with $K=2$

The idea to increase the number of shares is to use a line:

• Split and share keys

How to increase N , with $K=2$

The idea to increase the number of shares is to use a line:



- take the 2D plane with X and Y axis
- choose a random value R
- draw a line with slope R and passing through the point $(0, S)$
- the shares will be the points on the line $(1, S+R)$, $(2, S+2R)$, $(3, S+3R)$, ...
- Clearly it is possible to choose as many shares as wanted, since there's an infinite number of points on the line
- Given any two points on a line, it is possible to retrieve its equations using the interpolation, so $K=2$.
- Given just one point, it's impossible to retrieve any information about the line
- If we do this operation using the arithmetic modulo a large prime P , all we have said is still applicable.

How to increase K

Equation	Random parameters	Points needed to recover S
$(S + RX) \bmod P$	R	2
$(S + R_1X + R_2X^2) \bmod P$	R_1, R_2	3
$(S + R_1X + R_2X^2 + R_3X^3) \bmod P$	R_1, R_2, R_3	4

How to increase K

If we want to increase K, we can use functions that require more than two points to be defined.

For example, if we want $K=3$, we will use a quadratic function. In fact, three points are necessary to define uniquely a quadratic function. In this case, there will be two random parameters R_1 and R_2 , since the equation would be:

$$Y = R_2 X^2 + R_1 X + S$$

And we can go on like this increasing the polynomial degree.

For example, if we want $K=4$ we can use a cubic function with three random parameters and so on. Every curve use has infinite points, so we can also increase N at will.

Equation	Random parameters	Points needed to recover S
$(S + RX) \bmod P$	R	2
$(S + R_1X + R_2X^2) \bmod P$	R_1, R_2	3
$(S + R_1X + R_2X^2 + R_3X^3) \bmod P$	R_1, R_2, R_3	4

Example: Shamir's secret-sharing scheme

Suppose we want to use a (k, n) threshold scheme to share our secret S size

Assumed to be an element in a finite field F of size P where $0 < k \leq n < P$; $S < P$ and P is a prime number.

Secret Sharing

- A (k, n) threshold polynomial can be written by
$$s(x) \equiv M + s_1x + s_2x^2 + \dots + s_{k-1}x^{k-1} \pmod{p}$$
- Deliver $(x_i, s(x_i))$ to the i -th participant

p = a (large) prime number

M : secret value

$s_1, \dots, s_{k-1}$: randomly chosen from $[0, p-1]$

Secret Sharing...

- To reveal the secret value M , we must collect (at least k) $\geq k$ shadows.
- Without loss of generality, we use $(x_1, s(x_1)), \dots, (x_k, s(x_k))$ as k shadows.
- We can reveal the secret value M by using Lagrange interpolation.

$$s(x) \equiv \sum_{j=1}^k \left[s(x_j) \prod_{i=1, i \neq j}^k \frac{x - x_i}{x_j - x_i} \right] \bmod p$$

- where $M = s(0)$

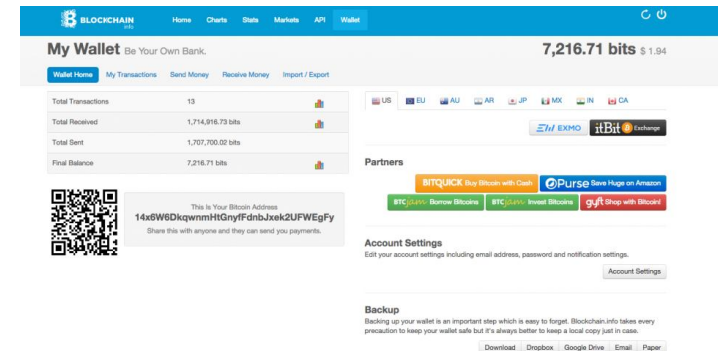
Secret sharing Pros and Cons:

it is possible to store the shares separately. So an adversary needs to be able to retrieve K shares in order to get back the secret key. So, he needs to break the security of different places K times. In addition, we can afford to lose some shares, since only K of them chosen randomly are needed to retrieve the key.

if we want to sign a transaction, we need to reconstruct the key bringing the shares together. This is a point of vulnerability. If we get attacked at this moment the adversary can recover the full key.

• Online wallets and exchanges

Online wallets



Advantages:

- it is not necessary to install software to use a web based wallet, or a simple app on the phone
- it works across multiple devices. Even if a device is lost, the wallet will still be available

Disadvantages:

- if the website or app is malicious or gets compromised somehow, the Bitcoins can be lost. It is necessary to trust the website as more secure than oneself.

- **Cryptocurrency exchanges**

Bitcoin exchanges are businesses that, at least from a user interface point of view, are similar to banks:

- accept deposits of Bitcoins and fiat currency (es. ₹, \$, €, ...)
- enable to send and receive Bitcoins
- buy and sell Bitcoins for fiat currency. Typically they try to match Bitcoin buyers and sellers. So they find some customer who wants to buy bitcoins with Rupees/dollars/euros and some other customer who wants to sell bitcoins for dollars/euros and try to match them up.

Exchange buy and sell process

- **Exchange risks**

It is really easy to trade back and forth from fiat currency and Bitcoins using an Exchange.

The risky aspect is that it behaves such as a bank, so:

- risk of **bank run**.
- **Ponzi scheme**
- risk of a **cyber attack**



- **Proof of reserve**

Proof of Reserve is a cryptographic technique that the exchange might use to boost consumer comfort. The goal is that a Bitcoin exchange can prove that it owns a fractional reserve. For instance, 25% of the deposits made by users might be obtainable for withdrawal.

There are two components to the Proof of Reserve:

- prove **how much reserve** it's holding.
- prove **how many demand deposits** the group holds.

To get the amount of **fractional reserve** divide those two numbers by themselves

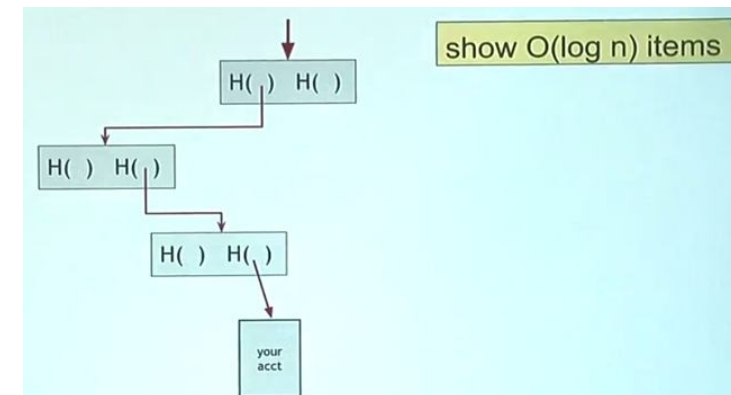
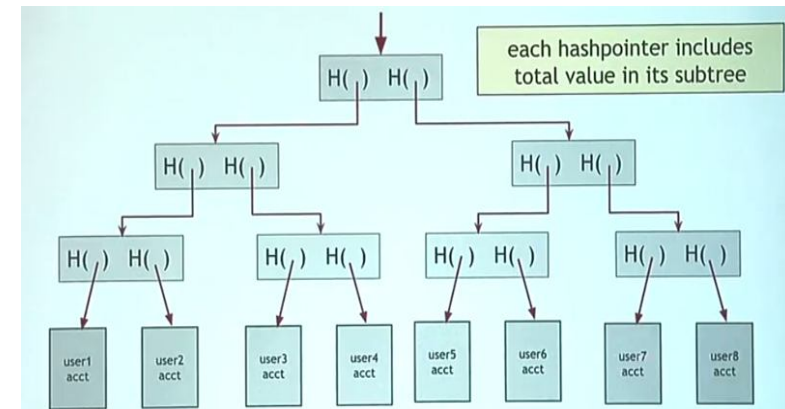
Prove the reserve amount

The company proves it owns a certain amount of BTC by doing the following:

- It creates a transaction that sends the claimed amount of BTC to itself (e.g., 100,000 BTC).
- This self-payment transaction uses the same private key that was used to sign a random string provided by an unbiased third party..
- This proves that the person with the private key is participating in the proof of reserve .
- It shows that the company or someone cooperating with the company controls at least the claimed amount of BTC.
- The company might prove ownership of 100,000 BTC even if it actually has more (e.g., 150,000 BTC).

Prove the number of deposits

- **Create a Merkle Tree:** Build a binary tree using hash pointers, which tell where the information is and its hash.
- **Label Pointers with Values:** Add a label to each hash pointer with the total Bitcoin value of everything under that pointer in the tree.
- **Place Customer Accounts:** Put all customer accounts at the bottom of the tree.
- **Combine Accounts:** Combine the accounts into a Merkle tree, so each node's hash pointer is labeled with the sum of the two pointers below it.
- **Root Represents Total Deposits:** The root of the tree will represent the total deposit amount.
- **Sign the Root:** The exchange signs the root of the tree to validate it.
- **Customer Verification:** Each customer can request to see their inclusion in the tree.
- **Show Path to Account:** The exchange shows the path to the customer's account.
- **Check Consistency:** Customers check that the hash pointers are consistent and that their deposit adds up to the total.
- **Verify All Branches:** If all customers do this, every branch of the tree is explored and verified.



Conclusion:

If the two pieces of the proof gets correctly verified, then the exchange has proven that:

- They own an amount of at least X Bitcoin of reserve
- The total deposits of the customers correspond to Y

So the reserve fraction would be X/Y , and it can be compared to what's declared by the company. These information can be independently verified by anybody, so it's a proof that can increase the trust in the exchange.

Payment Services:

Key Points

- Merchant's Motivation
- Risks for Merchants
- Role of Payment Services
- Process of Receiving Bitcoin Payments
 - Merchant Setup
 - Customer Payment Flow
- Settlement Process
- How Payment Services Manage Risks:

The screenshot shows a web interface titled "Choose A Way To Accept Bitcoin" with a link to "see examples of each payment method." Below the title are two rows of radio button options: "Type" with options "Button", "Hosted Page", "iFrame", and "Email Invoice"; and "Payment" with options "Buy now", "Donation", and "Subscription". Under the "Button" type, there are four button style previews: a grey button with "Pay with Bitcoin" and a Bitcoin icon, a grey button with "Pay with Bitcoin" and a Bitcoin icon, a blue button with a Bitcoin icon and "Pay With Bitcoin", and a blue button with a Bitcoin icon and "Pay With Bitcoin". Below these are input fields for "Item Name" (containing "Alpaca Socks"), "Amount" (set to "BTC" and "0.00"), and "Item Description" (containing "The ultimate in lightweight footwear"). There is also a "Send Funds To" dropdown menu set to "My Wallet (0.00 BTC)". A link "Show Advanced Options" is visible. At the bottom is a blue button labeled "Generate Button Code".

Example payment service interface for generating a pay-with-Bitcoin button.

- **Payment process**

when a customer chooses to pay with Bitcoin using the new button:

- the merchant delivers a page that contains the "**Pay with Bitcoin**" button. The button will contain a **transaction ID**, an identifier meaningful to the merchant in its own accounting system, along with an **amount** to be paid.
- The information that the button has been clicked will be sent to the payment service, along with the transaction id, the amount and merchant's identity.
- The payments service knows that the customer wants to pay with Bitcoins, so will start an interaction with the user to give information about how to pay.
- When the user confirms the payment, there will be a redirect that goes back to user's browser. This will also send a message to the merchant, saying that for the payment service everything is ok until then.
- Later, the payment service will directly send a confirmation to the merchant, when it's fully confirmed by the blockchain.
- The merchant can now send the product to the customer.

Transaction fees

Transaction fee= output value-input value

Current consensus fee:

A **Bitcoin transaction may not require a fee** under the following conditions:

- **Transaction Size:**
 - The total size of the transaction is **less than 1,000 bytes**.
- **Transaction Outputs:**
 - **All outputs** in the transaction are **at least 0.01 BTC** each.
- **Transaction Priority:**

The priority of the transaction is sufficiently high, calculated as:

- where **Priority** = $(\text{input_age}_1 * \text{input_value}_1 + \text{input_age}_2 * \text{input_value}_2 + \dots + \text{input_age}_N * \text{input_value}_N) / \text{transaction_size}$

input_value_in_base_units is the number of satoshis that the input is worth. One Bitcoin is worth 100,000,000 satoshis.

input_age is how many blocks the input has been present for. An unconfirmed transaction has an age of 0, and one that has 100 confirmations has an age of 100.

- a fee of 0.0001 Bitcoins every 1000 Bytes of the transaction. The transaction size is approximately 148 bytes per each input plus about 34 bytes per each output plus about 10 bytes for other information

So, even if these consensus rules are not followed, the transaction will probably find its way into the blockchain sooner or later. This is ok if you don't need it to be approved quickly.

Most online wallets and payment services include automatically a fee following those rules to forward reliable transactions. So you'll see a little bit of money racked off for transaction fees, when you make payments.